swapping them to disk, in order to have a larger virtual address space and give you the impression that it has more memory available than is actually installed on your machine.

In other words, caches are one of the foundations of modern IT systems. Of course, since fast storage is costlier, there is only a limited amount of it, and the majority of the data will have to be left out of the cache.

The algorithm used to decide what data stays in the cache determines the behavior of the cache and the rate of cache hit (when the data searched for are already in the cache) and miss. Some of the most used algorithms are *LRU* (**L**east **R**ecently **U**sed), *MRU* (**M**ost **R**ecently **U**sed) and *LFU* (**L**east **F**requently **U**sed). Chapter 6 gets into the details of these algorithms, but for now it is enough to mention that they, as well as many other cache replacement policies, all suffer "one-hit-wonders". In other words, they struggle with objects, memory locations or web pages requested just once, and never again (in the average lifetime of the cache). This is particularly common for routers and *Content Delivery Networks* (CDNs), were an average of 75% of the requests for a node are one-hit-wonders.

Using a dictionary to keep track of requests allows us to only store an object in cache when it's requested for the second time, filtering out one-hit-wonders and improving the cache *hit ratio*. Bloom filters allow performing such lookups using amortized constant time operations and with limited space, at the cost of accepting some painless false positive: for this application, the only result of false positives, however, would be a tiny reduction of the cache